Supplementary Document: Modified GUESS Approach Using Machine Decision-makers for an Efficient Interactive Multi-criterion Decision-Making Procedure

Deepanshu Yadav
Michigan State University
East Lansing, USA
deepansh@msu.edu

Kalyanmoy Deb
Michigan State University
East Lansing, USA
kdeb@egr.msu.edu

***Abstract*—This is a supplementary document (SD) to the paper-'*Modified GUESS Approach with Machine Decision-maker for an Efficient Interactive Multi-criterion Decision-Making Procedure*'. This SD details the concept of Machine Learning (ML)-based Decision-maker (DM) for interactive Multi-criterion Decision-Making (iMCDM). This SD provide additional details on the key components of Machine-DM. In particular, this SD provides a detailed description of Class Predictor ML (ANN1), Parameter Predictor ML (ANN2), computation of bounding parameters ($\bar{z}, \varepsilon$), ML parameters used to train ANN1 and ANN2 and associated error metrics.**

***Index Terms*—Interactive Multi-criteria Decision-making, Machine Learning, Evolutionary Multi-objective optimization, Preference Learning, Benchmarking.**

I. Introduction

EVOLUTIONARY multi-objective optimization (EMO) algorithms have been primarily focused on obtaining a well-distributed and diverse set of Pareto-optimal (PO) solutions [1]–[3]. In this paper modified GUESS approach is proposed and its comparison with existing GUESS approach [4] has been conducted. However, this requires a benchmarking task involving human decision makers (DMs). Benchmarking interactive multi-criteria decision-making (iMCDM) procedures is inherently challenging due to several factors. First, the direct involvement of a human decision-maker (DM) introduces subjectivity, and the systematic emulation of human DM behavior has received limited attention in the iMCDM literature. Second, the choice of the achievement scalarizing function (ASF) poses an additional challenge, as ASF formulations vary across iMCDM methods and depend on DM-provided preferences, which themselves are uncertain and inconsistent. In light of these challenges, the main manuscript to this supplementary document (SD) proposes a machine learning (ML)-based Machine-DM to emulate human decision-making and preference articulation within the iMCDM framework.

The rest of the supplementary document (SD) is organized as follows. Section II provides a details description on Class Predictor ML: ANN2. Detail description on Parameter Predictor ML is provided in Section III. Section IV provides detailed computational framework for generating data to train ANNs. The information on ANN architecture and ANN error metric for problems used in main manuscript is provided in Section V. Finally, conclusions are provided in Section VI.

II. Details on Class Predictor ML: ANN1

Ideally, the class-predictor illustrated in Figure 1 is an ANN that takes a decision vector ($\mathbf{x}$) as input and outputs the corresponding objective classification status vector:

$$\mathcal{F}_{CL} : \mathbf{x} \xrightarrow{\mathcal{F}_{CL}} \{I^{<}, I^{\leq}, I^{=}, I^{\geq}\}^M, \quad \mathbf{x} \in \mathbb{R}^n.$$

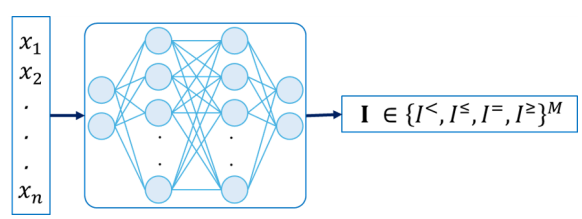

Fig. 1: Class predictor ML ($\mathcal{F}_{CL}$): $\mathbf{x}(\in \mathbb{R}^n) \rightarrow \mathbf{I}(\in \mathbb{Z}^M)$. $\mathbb{Z} = \{1, 2, 3, 4\}$ represents four Classes $\{I^{<}, I^{\leq}, I^{=}, I^{\geq}\}$.

In essence, $\mathcal{F}_{CL}$ establishes a direct mapping from a decision vector ($\mathbf{x}$) to its objective classification status ($\mathbf{I}$) without explicitly computing the objective values. A key advantage of $\mathcal{F}_{CL}$ is that it requires no prior knowledge of the pessimistic point (R), the target solution (T), or the actual objective values (refer to Figure 2). Consequently, using this ANN for a given $\mathbf{x}$ eliminates the need for costly objective function evaluations. In this sense, ANN1 serves both as a surrogate model and as a classifier, providing the objective classification.

III. Details on Parameter Predictor ML: ANN2

The parameter-predictor ANN (Figure 3) estimates the bounding parameters ($\bar{z}_i, \varepsilon_j; i \in I^{\leq}, j \in I^{\geq}$), which define the lower and upper bounds of objective values for the subsequent iteration of the iMCDM procedure. Traditionally,

TABLE I: Classification status for two-objective PF solutions presented in Figure 2.

Class $\mathbf{I}$	f_1 Range	f_2 Range	Representative PO-Point
$[I^{<}, I^{\geq}]$	$> f_1^R$	$< f_2^T - 0.5\delta_2$	A
$[I^{\leq}, I^{\geq}]$	$[f_1^T + 0.5\delta_1, f_1^R]$	$< f_2^T - 0.5\delta_2$	B
$[I^{=}, I^{=}]$	$[f_1^T - 0.5\delta_1, f_1^T + 0.5\delta_1]$	$[f_2^T - 0.5\delta_2, f_2^T + 0.5\delta_2]$	C
$[I^{\geq}, I^{\leq}]$	$< f_1^T - 0.5\delta_1$	$[f_2^T + 0.5\delta_2, f_2^R]$	D
$[I^{\geq}, I^{<}]$	$< f_1^T - 0.5\delta_1$	$> f_2^R$	E

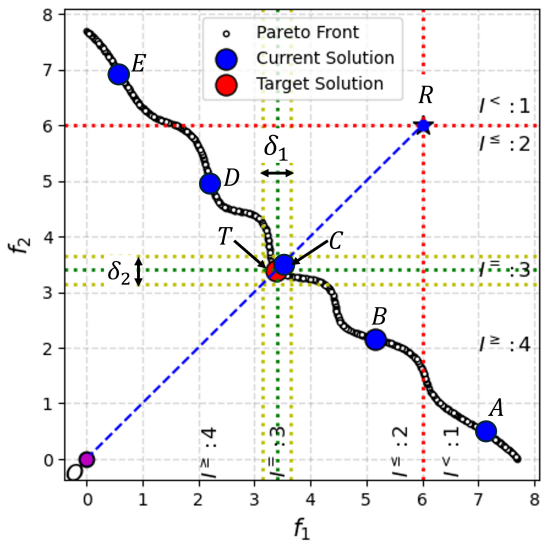

Fig. 2: Background on the Machine-DM approach. A-E are Pareto-optimal points located in different classes. R and O indicate a pessimistic point and ideal point, respectively. δ_1 and δ_2 are the width of Class $I^{=}$ and T is the target solution.

TABLE II: Problem description and parameter setting for GA and Machine-DM. Parameters M, n, and J are number of objectives, variables, and constraints, respectively.

Examples	Problem Details			Machine-DM Parameters		ANN1/ANN2 Parameters		
	M	n	J	$\hat{\delta}$	$\hat{\alpha}$	lr_1	lr_2	l_{reg}
ZDT3	2	5	0	0.100	0.100	0.0100	0.010	0.010
Knee	2	5	0	0.100	0.075	0.0100	0.010	0.010
Knee	3	5	0	1 0.050	0.100	0.0010	0.001	0.010
Crash.	3	5	0	0.100	0.050	0.0010	0.002	0.020
Car Side	3	7	10	0.100	0.075	0.0020	0.001	0.012
C2-DTLZ2	3	10	1	0.100	0.100	0.0001	0.0001	0.001
DTLZ7	3	10	0	0.100	0.075	0.001	0.001	0.001
River	4	2	0	0.015	0.025	0.0025	0.001	0.010
WATER	5	3	7	0.025	0.035	0.0010	0.002	0.020
DTLZ2	8	10	0	0.050	0.200	0.0010	0.001	0.010

these bounds are provided manually by a human decision-maker. The parameter-predictor ANN ($\mathcal{F}_{PP}$) automates this process by directly inferring the bounding parameters from a given decision vector $\mathbf{x}$:

$$\mathcal{F}_{PP} : \mathbf{x} \xrightarrow{\mathcal{F}_{PP}} \{\bar{z}, \varepsilon, 0\}^M.$$

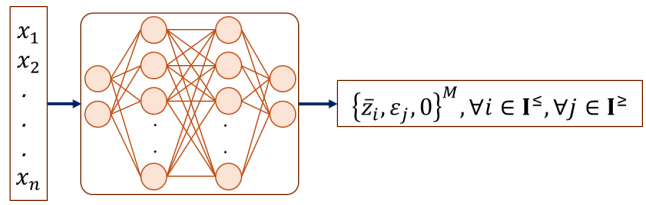

Fig. 3: Parameter predictor ML ($\mathcal{F}_{PP}$) : $\mathbf{x} \rightarrow \{\bar{z}, \varepsilon, 0\}^M$.

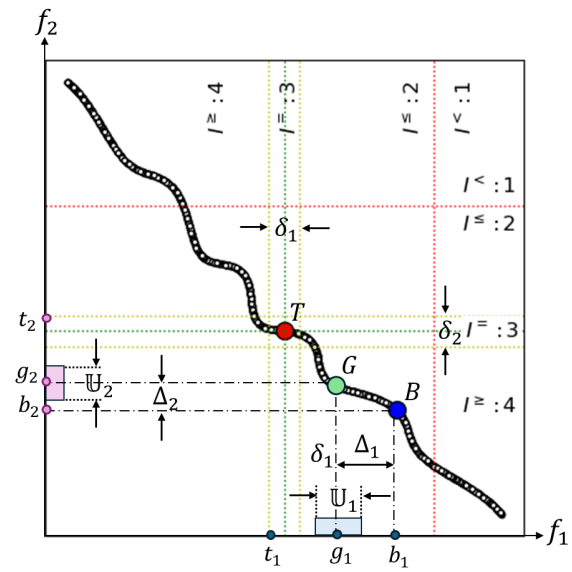

Fig. 4: Defining the bounding parameters $(\bar{z}_i, \varepsilon_j)$ for parameter-predictor ML.

Here, ε_j corresponds to each Class $I^{\geq}$ objective, $\bar{z}i$ corresponds to each Class $I^{\leq}$ objective, and zero is assigned for objectives in Classes $I^{<}$ or $I^{=}$. Specifically, ε_j represents the upper bound for the j-th Class $I^{\geq}$ objective (as identified by $\mathcal{F}_{CL}$), ensuring $f_j(\mathbf{x}) \leq \varepsilon_j$, while $\bar{z}_i$ is the lower bound for the i-th Class $I^{\leq}$ objective, requiring $f_i(\mathbf{x}) \geq \bar{z}i$. For objectives in Classes $I^{<}$ and $I^{=}$, no bounds are necessary, so $\mathcal{F}_{PP}$ outputs zero.

For a Class $I^{\geq}$ objective, the upper bound ε_j is determined based on the current objective value $f_j^c = f_j(\mathbf{x}^{*,c})$ and the perceived target f_j^T. When f_j^c is far from f_j^T, the decision-maker may be uncertain about the desired improvement. This uncertainty is addressed by computing Δ_j as shown in Figure 4 and defined in Equation (1):

$$\Delta_j = \min\left[\alpha_j\left(z_j^{\text{nad}} - z_j^*\right), \left(f_j^T - f_j^c\right) + 0.5\delta_j\right], \quad (1)$$

TABLE III: Error metric for ANN models across benchmark MCDM problems and instances.

S.I.	Example	Objective function	ANN1 ($\mathcal{F}_{CL}$) Error Metric				ANN2 ($\mathcal{F}_{PP}$) Error Metric	
			Accuracy	Precision	Recall	F1-Score	R^2	MSE
1	ZDT3 (bi-objective)	f_1	1.0000	1.0000	1.0000	1.0000	0.9962	0.0002
		f_2	0.9429	0.9167	0.9474	0.9206	0.9566	0.0056
2	Knee (two-objective)	f_1	1.0000	1.0000	1.0000	1.0000	0.9956	0.0134
		f_2	1.0000	1.0000	1.0000	1.0000	0.9965	0.0108
3	Knee (three-objective)	f_1	0.9554	0.9482	0.9296	0.9372	0.9930	0.0198
		f_2	0.9732	0.9516	0.9609	0.9550	0.9953	0.0133
		f_3	0.9286	0.8947	0.9023	0.8983	0.9780	0.0321
4	Vehicle Crashworthiness (three-objective)	f_1	0.9610	0.9495	0.9518	0.9494	0.9962	0.0026
		f_2	0.9610	0.9651	0.9453	0.9522	0.9915	0.0066
		f_3	1.0000	1.0000	1.0000	1.0000	0.9975	0.0023
5	Car side Impact (three-objective)	f_1	0.9505	0.9259	0.9489	0.9364	0.9952	0.0049
		f_2	0.9286	0.9059	0.8633	0.8752	0.9973	0.0028
		f_3	0.9670	0.9468	0.8982	0.9201	0.9950	0.0042
6	C2-DTLZ2 (three-objective)	f_1	0.9882	0.9696	0.9622	0.9658	0.9898	0.0253
		f_2	1.0000	1.0000	1.0000	1.0000	0.9889	0.01265
		f_3	0.9704	0.9464	0.9511	0.9450	0.9927	0.0219
7	DTLZ7 (three-objective)	f_1	0.9908	0.9837	0.9659	0.9933	0.9966	0.0002
		f_2	0.9662	0.9656	0.9380	0.9512	0.9959	0.0003
		f_3	0.9415	0.8233	0.7799	0.7985	0.9863	0.0049
8	River Pollution (four-objective)	f_1	0.9950	0.9947	0.9976	0.9961	0.9987	0.0003
		f_2	0.9750	0.9421	0.9647	0.9514	0.9939	0.0001
		f_3	0.9600	0.9681	0.9059	0.9447	0.9954	0.0196
		f_4	0.9650	0.9748	0.9359	0.9527	0.9931	0.0535
9	WATER (five-objective)	f_1	0.9600	0.8344	0.9132	0.8605	0.9992	0.0008
		f_2	0.9475	0.8056	0.8211	0.8130	0.9996	0.0004
		f_3	0.9800	0.7417	0.7367	0.7391	0.9997	0.0003
		f_4	0.9775	0.7606	0.8051	0.7163	0.9994	0.0004
		f_5	0.9425	0.7073	0.7137	0.7013	0.9988	0.0012
10	DTLZ2 (eight-objective)	f_1	0.9503	0.9057	0.9121	0.9080	0.9830	0.0478
		f_2	0.9526	0.9184	0.9159	0.9161	0.9756	0.0625
		f_3	0.9464	0.9364	0.8994	0.9165	0.9786	0.0601
		f_4	0.9619	0.9382	0.9199	0.9278	0.9786	0.0677
		f_5	0.9611	0.9513	0.9603	0.9553	0.9882	0.0383
		f_6	0.9580	0.9358	0.9244	0.9299	0.9860	0.0438
		f_7	0.9565	0.9430	0.9249	0.9319	0.9845	0.0581
		f_8	0.9775	0.9727	0.9743	0.9734	0.9926	0.0222

where α_j is a problem-specific parameter, and δ_j defines the Class $I^{=}$ range, as illustrated in Figure 4. The upper bound ε_j is then set with a small uniform randomness ($\mathbb{U}$) to reflect human uncertainty:

$$\varepsilon_j = f_j^c + (1 + \alpha_j \mathbb{U}[-1,\ 1])\Delta_j, \tag{2}$$

ensuring that $\varepsilon_j > f_j^c$ when $\alpha_j < 1$. Similarly, for Class $I^{\leq}$ objectives, the lower bound $\bar{z}_i$ is computed as:

$$\bar{z}_i = f_i^c - (1 + \alpha_i \mathbb{U}[-1,\ 1])\Delta_i, \tag{3}$$

with Δ_i defined analogously to Equation (1) using index i. This ensures $\bar{z}_i < f_i^c$ for $\alpha_i < 1$. The inclusion of $\mathbb{U}$ introduces variability in the bounds, mimicking the inexact responses of a human DM.

IV. Data Generation for Training ANNs

In this section, we provide a comprehensive computational framework for generating data for training ANN1 and ANN2 in detail. For a given M-objective and n-variable problem, generate a well distributed, well-converged, and a diverse set of near Pareto optimal solutions $\left(\left[\mathbf{x}^*, \mathbf{f}^*\right]_{i=1}^{N}\right)$. Consider that $R = \mathbf{f}^R$ is an unknown M-dimensional pessimistic point and $T = \mathbf{f}^T$ is the M-dimensional target point. Machine-DM parameters described in main manuscript are δ and α. Also, $\mathbf{z}^*$ and $\mathbf{z}^{nad}$ represent ideal and nadir point, respectively.

A. Data Generation for ANN1

For a M-dimensional PO point $\mathbf{f}^*$, the M-dimensional objective class $\mathbf{I}$ is computed as follows:

$$I_m = \begin{cases} 1 : I^{<}, & \text{if} \quad f_m^* > f_m^R \\ 2 : I^{\leq}, & \text{if} \quad f_m^R \geq f_m^* > f_m^T + 0.5 \times \hat{\delta}_m \\ 3 : I^{=}, & \text{if} \quad f_m^T - 0.5 \times \hat{\delta}_m \geq f_m^* \geq f_m^T \\ & \quad\quad +0.5 \times \hat{\delta}_m \\ 4 : I^{\geq}, & \text{if} \quad f_m^* < f_m^T - 0.5 \times \hat{\delta}_m \end{cases} \quad m = 1, \ldots, M. \tag{4}$$

Here, $\hat{\delta}_m = \delta_m \times (z_m^{nad} - z_m^*)$ defines the width of target objective class ($I^{=}$) in m-th objective. Next, for all the PO solutions $\left(\left[\mathbf{x}^*, \mathbf{f}^*\right]_{i=1}^N\right)$ obtain the class according to Equation (4). Get the training data ($\mathcal{D}_{ANN1}$) for ANN1 as follows:

$$\mathcal{D}_{ANN1} = \left[\mathbf{x}^{*,(i)}, \mathbf{I}^{(i)}\right]_{i=1}^N \tag{5}$$

Next, the data $\mathcal{D}_{ANN1}$ is used for train-test-validation split and for training ANN1 ($\mathcal{F}_{CL}$).

B. Data Generation for ANN2

In this section, we provide the computational framework for generating training data for ANN2 ($\mathcal{F}_{PP}$). Following the objective classification definition provided in Equation (4), compute the M-dimensional bounding parameter ($\mathbf{\Lambda}$) as follows:

$$\Lambda_m = \begin{cases} 0 & \text{if } I_m \in \mathbf{I}^{<} \\ \bar{z}_m, & \text{if } I_m \in \mathbf{I}^{\leq} \\ 0 & \text{if } I_m \in \mathbf{I}^{=} \\ \varepsilon_m, & \text{if } I_m \in \mathbf{I}^{>} \end{cases}; \quad m = 1, \ldots, M. \tag{6}$$

The detailed computation for $\bar{z}$ and ε is provided in Section III. Next, for all the PO solutions $\left(\left[\mathbf{x}^*, \mathbf{f}^*\right]_{i=1}^N\right)$, compute the bounding parameter vector ($\mathbf{\Lambda}$). Get the training data ($\mathcal{D}_{ANN2}$) as follows:

$$\mathcal{D}_{ANN2} = \left[\mathbf{x}^{*,(i)}, \mathbf{\Lambda}^{(i)}\right]_{i=1}^N. \tag{7}$$

Next, the data $\mathcal{D}_{ANN2}$ is used for train-test-validation split and for training ANN2 ($\mathcal{F}_{PP}$).

V. Architecture and Error Metric for ANNs

The architecture of ANN1 and ANN2 is provided in Table IV and Table V, respectively. For ANN1 and ANN2, four-layered feed-forward neural networks are used with a reduced number of hidden layers [128, 64, 32, 16] and A*dam optimizer* [5] with a learning rate set to lr_1 and lr_2, respectively. ANN1 uses *Softmax* and ANN2 uses *Sigmoid* activation functions. For ANN2, L2-regularization with parameter l_{reg} is used to train the respective ANN. For both the ANNs, 80% of data is used in training and 20% data is used for testing. In training phase, 20% of training data is used for cross-validation purpose. The batch-size and maximum epoch for both the ANNs are set to 32 and 500, respectively. For training ANN1, error metrics – Accuracy, Precision, Recall, and F-1 Score – are checked for their closeness to one. For training ANN2, R^2 and MSE is used as error metric.

TABLE IV: A generic architecture of ANN1–Class Predictor ANN ($\mathcal{F}_{CL}$) for an n-variable, M-objective MOO problem, trained with fixed hidden (dense) layer [128,64,32,16].

Layer Type	Output Shape	Number of Parameters	Connected to
Input Layer	n	0	—
Dense Layer 1	128	$128(n+1)$	Input Layer
Dense Layer 2	64	8,256	Dense Layer 1
Dense Layer 3	32	2,080	Dense Layer 2
Dense Layer 4	16	528	Dense Layer 3
Output Layer 1	1	68	Dense Layer 4
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Output Layer M	1	68	Dense Layer 4
Total Trainable Parameters:		$128n + 10,992 + 68M$	

TABLE V: A generic architecture of ANN2–Parameter Predictor ANN ($\mathcal{F}_{PP}$) for an n-variable, M-objective MOO problem, trained with fixed hidden (dense) layer [128,64,32,16].

Layer Type	Output Shape	Number of Parameters	Connected to
Input Layer	n	0	—
Dense Layer 1	128	$128(n+1)$	Input Layer
Dense Layer 2	64	8,256	Dense Layer 1
Dense Layer 3	32	2,080	Dense Layer 2
Dense Layer 4	16	528	Dense Layer 3
Output Layer 1	1	17	Dense Layer 4
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Output Layer M	1	17	Dense Layer 4
Total Trainable Parameters:		$128n + 10,992 + 17M$	

VI. Conclusions

This paper has proposed two efficient variants of GUESS method– GUESS-I and GUESS-II. Using Machine-DM, three Machine-DM based procedures-MachGUESS (original), MachDM-GUESS-I (modified), and MachDM-GUESS-II (modified) has been developed. Then these three MachDM-iMCDM procedures have been statistically compared using four newly introduced performance metrics using Wilcoxon Signed-rank Test [6]. Implementation on six benchmark and four real-world example reveals that modified GUESS-I method is the most efficient among three approaches.

References

[1] K. Deb, *Multi-objective optimization using evolutionary algorithms*. Chichester, UK: Wiley, 2001.

[2] C. A. C. Coello, G. B. Lamont, and D. A. V. Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, ser. Genetic and Evolutionary Computation. New York, NY: Springer, 2007.

[3] K. Miettinen, *Nonlinear multiobjective optimization*. Springer Science & Business Media, 2012, vol. 12.

[4] J. T. Buchanan, "A naive approach for solving MCDM problems: The GUESS method," *Journal of the Operational Research Society*, vol. 48, no. 2, pp. 202–206, 1997.

[5] I. K. M. Jais and A. R. Ismail, "Adam optimization algorithm for wide and deep neural network," *Knowledge Engineering and Data Science*, vol. 2, no. 1, p. 10, 2019.

[6] T. Harris and J. W. Hardin, "Exact Wilcoxon signed-rank and Wilcoxon Mann–Whitney ranksum tests," *The Stata Journal*, vol. 13, no. 2, pp. 337–343, 2013.